

Uniwersytet Wrocławski

Rafał Lewandków

PROGRAMOWANIE URZĄDZEŃ MOBILNYCH 2

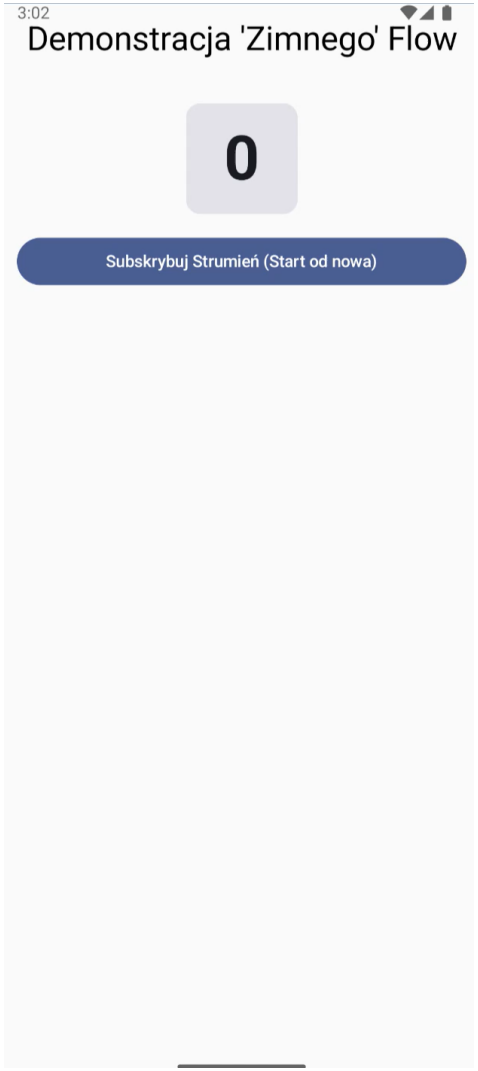
WYKŁAD 5

Reaktywne Zarządzanie Stanem:

- Flow
- StateFlow
- SharedFlow

Flow

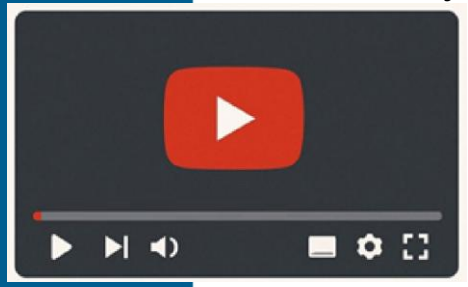
Do tej pory nasze funkcje suspend zwracały **pojedynczą wartość** (np. String lub List). A co, jeśli chcemy, aby funkcja zwracała **sekwencję wartości** w czasie? Do tego właśnie służy **Flow** – **asynchroniczny strumień danych**.



Uniwersytet Wrocławski

Flow

Do tej pory nasze funkcje suspend zwracały **pojedynczą wartość** (np. String lub List). A co, jeśli chcemy, aby funkcja zwracała **sekwencję wartości** w czasie? Do tego właśnie służy **Flow** – **asynchroniczny strumień danych**.



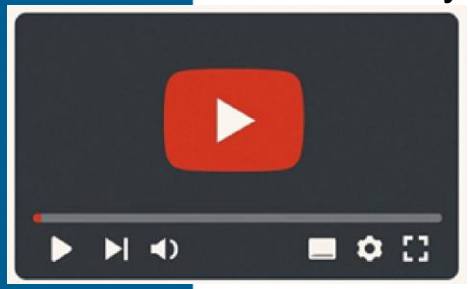
To jest film na kanale YouTube o tytule numberStream i istnieje jako gotowy do odtworzenia materiał. Samo jego istnienie **nic nie robi** – nikt go jeszcze nie ogląda. Jest ***"zimny".***

```
fun numberStream(): Flow<Int> = flow {
    println("Flow: Strumień wystartował!") // To wydrukuje się przy każdej
    for (i in 1..3) {
        delay(1000)
        emit(i) // 'emit()' wysyła wartość do strumienia
    }
}

// Uruchomienie wewnątrz korutyny
scope.launch {
    println("Zaczynam słuchać strumienia...")
    numberStream().collect { number ->
        println("Odebrano liczbę: $number")
    }
    println("Strumień zakończony.")
}
```

Flow

Do tej pory nasze funkcje suspend zwracały **pojedynczą wartość** (np. String lub List). A co, jeśli chcemy, aby funkcja zwracała **sekwencję wartości** w czasie? Do tego właśnie służy **Flow** – **asynchroniczny strumień danych**.



To jest film na kanale YouTube o tytule numberStream i istnieje jako gotowy do odtworzenia materiał. Samo jego istnienie **nic nie robi** – nikt go jeszcze nie ogląda. Jest ***"zimny".***

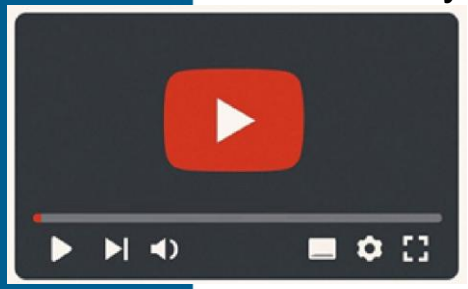
To jest treść filmu

```
fun numberStream(): Flow<Int> = flow {
    println("Flow: Strumień wystartował!") // To wydrukuje się przy każdej
    for (i in 1..3) {
        delay(1000)
        emit(i) // 'emit()' wysyła wartość do strumienia
    }
}

// Uruchomienie wewnątrz korutyny
scope.launch {
    println("Zaczynam słuchać strumienia...")
    numberStream().collect { number ->
        println("Odebrano liczbę: $number")
    }
    println("Strumień zakończony.")
}
```

Flow

Do tej pory nasze funkcje suspend zwracały **pojedynczą wartość** (np. String lub List). A co, jeśli chcemy, aby funkcja zwracała **sekwencję wartości** w czasie? Do tego właśnie służy **Flow** – **asynchroniczny strumień danych**.



To jest film na kanale YouTube o tytule numberStream i istnieje jako gotowy do odtworzenia materiał. Samo jego istnienie **nic nie robi** – nikt go jeszcze nie ogląda. Jest ***"zimny".***

To jest treść filmu

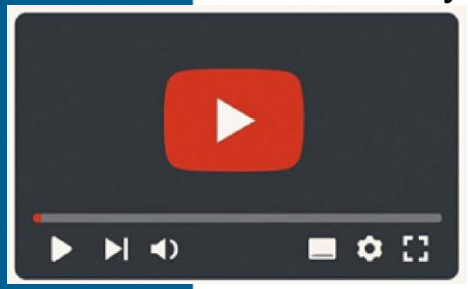
Co sekundę na ekranie pojawia się nowa liczba. emit to akt ***"wyemitowania"*** klatki filmu.

```
fun numberStream(): Flow<Int> = flow {
    println("Flow: Strumień wystartował!") // To wydrukuje się przy każdej
    for (i in 1..3) {
        delay(1000)
        emit(i) // 'emit()' wysyła wartość do strumienia
    }
}

// Uruchomienie wewnątrz korutyny
scope.launch {
    println("Zaczynam słuchać strumienia...")
    numberStream().collect { number ->
        println("Odebrano liczbę: $number")
    }
    println("Strumień zakończony.")
}
```

Flow

Do tej pory nasze funkcje suspend zwracały **pojedynczą wartość** (np. String lub List). A co, jeśli chcemy, aby funkcja zwracała **sekwencję wartości** w czasie? Do tego właśnie służy **Flow** – **asynchroniczny strumień danych**.



To jest film na kanale YouTube o tytule numberStream i istnieje jako gotowy do odtworzenia materiał. Samo jego istnienie **nic nie robi** – nikt go jeszcze nie ogląda. Jest ***"zimny".***

To jest treść filmu

Co sekundę na ekranie pojawia się nowa liczba. emit to akt ***"wyemitowania"*** klatki filmu.

Załadowanie linku z interesującym nas filmem.

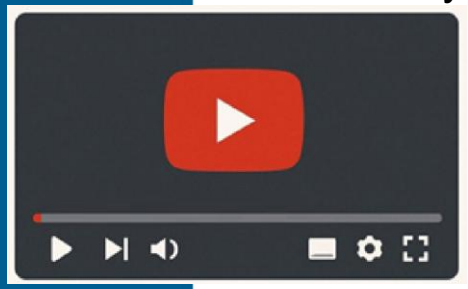
```
fun numberStream(): Flow<Int> = flow {
    println("Flow: Strumień wystartował!") // To wydrukuje się przy każdej
    for (i in 1..3) {
        delay(1000)
        emit(i) // 'emit()' wysyła wartość do strumienia
    }
}

// Uruchomienie wewnątrz korutyny
scope.launch {
    println("Zaczynam słuchać strumienia...")
    numberStream().collect { number ->
        println("Odebrano liczbę: $number")
    }
    println("Strumień zakończony.")
}
```

Uniwersytet Wrocławski

Flow

Do tej pory nasze funkcje suspend zwracały **pojedynczą wartość** (np. String lub List). A co, jeśli chcemy, aby funkcja zwracała **sekwencję wartości** w czasie? Do tego właśnie służy **Flow** – **asynchroniczny strumień danych**.



To jest film na kanale YouTube o tytule numberStream i istnieje jako gotowy do odtworzenia materiał. Samo jego istnienie **nic nie robi** – nikt go jeszcze nie ogląda. Jest ***"zimny".***

To jest treść filmu

Co sekundę na ekranie pojawia się nowa liczba. emit to akt ***"wyemitowania"*** klatki filmu.

Załadowanie linku z interesującym nas filmem.

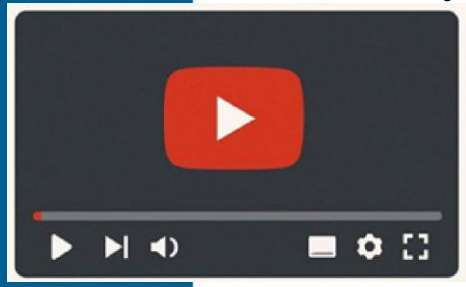
To jest moment, w którym widz wciska przycisk ***"Play„.***

```
fun numberStream(): Flow<Int> = flow {
    println("Flow: Strumień wystartował!") // To wydrukuje się przy każdej
    for (i in 1..3) {
        delay(1000)
        emit(i) // 'emit()' wysyła wartość do strumienia
    }
}

// Uruchomienie wewnątrz korutyny
scope.launch {
    println("Zaczynam słuchać strumienia...")
    numberStream().collect { number ->
        println("Odebrano liczbę: $number")
    }
    println("Strumień zakończony.")
}
```

Flow

Do tej pory nasze funkcje suspend zwracały **pojedynczą wartość** (np. String lub List). A co, jeśli chcemy, aby funkcja zwracała **sekwencję wartości** w czasie? Do tego właśnie służy **Flow** – **asynchroniczny strumień danych**.



To jest film na kanale YouTube o tytule numberStream i istnieje jako gotowy do odtworzenia materiał. Samo jego istnienie **nic nie robi** – nikt go jeszcze nie ogląda. Jest ***"zimny".***

To jest treść filmu

Co sekundę na ekranie pojawia się nowa liczba. emit to akt ***"wyemitowania"*** klatki filmu.

```
fun numberStream(): Flow<Int> = flow {
    println("Flow: Strumień wystartował!") // To wydrukuje się przy każdej
    for (i in 1..3) {
        delay(1000)
        emit(i) // 'emit()' wysyła wartość do strumienia
    }
}

// Uruchomienie wewnątrz korutyny
scope.launch {
    println("Zaczynam słuchać strumienia...")
    numberStream().collect { number ->
        println("Odebrano liczbę: $number")
    }
    println("Strumień zakończony.")
```

Załadowanie linku z interesującym nas filmem.

To jest moment, w którym widz wciska przycisk ***"Play„.***

Gdyby inny "*widz*" (inna korutyna) również wywołał numberStream().collect(), obejrzałby **ten sam film od początku**, we własnym tempie. To jest esencja "***zimnego***" strumienia Flow.

Uniwersytet Wrocławski

Flow

Do tej pory nasze funkcje suspend zwracały **pojedynczą wartość** (np. String lub List). A co, jeśli chcemy, aby funkcja zwracała **sekwencję wartości** w czasie? Do tego właśnie służy **Flow** – **asynchroniczny strumień danych**.

```
// Definicja "zimnego" strumienia - to jest tylko przepis.
val coldStream: Flow<String> = flow {
    println("=> PRZEPIS: Rozpoczynam przygotowanie...")
    delay(100)
    emit("Składnik A")
    delay(100)
    emit("Składnik B")
    println("=> PRZEPIS: Zakończono.")
}

fun main() = runBlocking {
    println("--- PIECZENIE #1 ---")
    // Pierwsza osoba "piecze ciasto" (subskrybuje strumień)
    coldStream.collect { skladnik ->
        println("Kucharz 1 otrzymał: $skladnik")
    }

    println("\n--- PIECZENIE #2 ---")
    // Druga osoba "piecze ciasto" od nowa
    coldStream.collect { skladnik ->
        println("Kucharz 2 otrzymał: $skladnik")
    }
}
```

Flow

Do tej pory nasze funkcje suspend zwracały **pojedynczą wartość** (np. String lub List). A co, jeśli chcemy, aby funkcja zwracała **sekwencję wartości** w czasie? Do tego właśnie służy **Flow** – **asynchroniczny strumień danych**.

```
// Definicja "zimnego" strumienia - to jest tylko przepis.
val coldStream: Flow<String> = flow {
    println("=> PRZEPIS: Rozpoczynam przygotowanie...")
    delay(100)
    emit("Składnik A")
    delay(100)
    emit("Składnik B")
    println("=> PRZEPIS: Zakończono.")
}

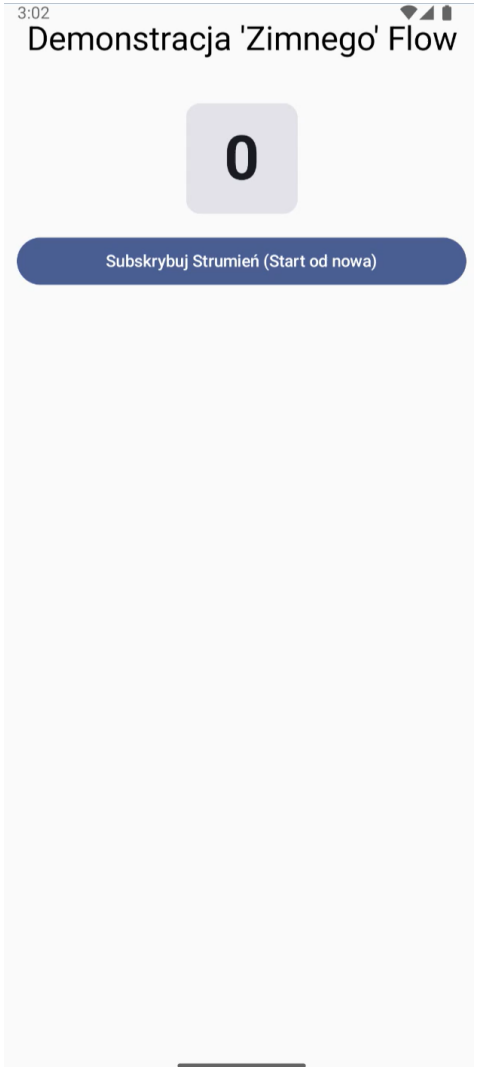
fun main() = runBlocking {
    println("--- PIECZENIE #1 ---")
    // Pierwsza osoba "piecze ciasto" (subskrybuje strumień)
    coldStream.collect { skladnik ->
        println("Kucharz 1 otrzymał: $skladnik")
    }

    println("\n--- PIECZENIE #2 ---")
    // Druga osoba "piecze ciasto" od nowa
    coldStream.collect { skladnik ->
        println("Kucharz 2 otrzymał: $skladnik")
    }
}
```

```
--- PIECZENIE #1 ---
=> PRZEPIS: Rozpoczynam przygotowanie...
Kucharz 1 otrzymał: Składnik A
Kucharz 1 otrzymał: Składnik B
=> PRZEPIS: Zakończono.

--- PIECZENIE #2 ---
=> PRZEPIS: Rozpoczynam przygotowanie...
Kucharz 2 otrzymał: Składnik A
Kucharz 2 otrzymał: Składnik B
=> PRZEPIS: Zakończono.
```

Flow




```
@Composable
fun FlowDemoScreen() {
    val scope = rememberCoroutineScope()
    val logs = remember { mutableStateListOf<String>() }
    var latestValue by remember { mutableStateOf( value = 0) }
    val listState = rememberLazyListState()

    LaunchedEffect( key1 = logs.size) {
        if (logs.isNotEmpty()) {
            listState.animateScrollToItem( index = logs.size - 1)
        }
    }

    Column(...) {
        Text(...)
        Spacer(Modifier.height( height = 20.dp))

        Card(modifier = Modifier.padding( all = 16.dp)) {...}

        Button(
            onClick = {
                scope.launch {
                    logs.add("--- NOWA SUBSKRYPCJA ---")
                    numberStream(logs).collect { number ->
                        latestValue = number
                    }
                }
            },
            modifier = Modifier.fillMaxWidth()
        ) {...}
```

Uniwersytet Wrocławski

Flow

Obiekt **stanu**, który pozwala programowo kontrolować pozycję przewijania LazyColumn

```
@Composable
fun FlowDemoScreen() {
    val scope = rememberCoroutineScope()
    val logs = remember { mutableStateListOf<String>() }
    var latestValue by remember { mutableStateOf( value = 0) }
    val listState = rememberLazyListState()

    LaunchedEffect( key1 = logs.size) {
        if (logs.isNotEmpty()) {
            listState.animateScrollToItem( index = logs.size - 1)
        }
    }

    Column(...) {
        Text(...)
        Spacer(Modifier.height( height = 20.dp))

        Card(modifier = Modifier.padding( all = 16.dp)) {...}

        Button(
            onClick = {
                scope.launch {
                    logs.add("--- NOWA SUBSKRYPCJA ---")
                    numberStream(logs).collect { number ->
                        latestValue = number
                    }
                }
            },
            modifier = Modifier.fillMaxWidth()
        ) {...}
```

Flow

Obiekt **stanu**, który pozwala programowo kontrolować pozycję przewijania LazyColumn

LaunchedEffect służy do uruchamiania **efektów ubocznych** (takich jak animacje, wywołania sieciowe, itp.) w odpowiedzi na **zmiany stanu** lub pojawienie się komponentu na ekranie.
W tym konkretnym przypadku rozwiązuje on następujący problem: gdy do listy logs dodawane są nowe wpisy, mogą one pojawić się na dole, poza widocznym obszarem LazyColumn. Użytkownik musiałby ręcznie przewijać listę, aby zobaczyć najnowsze logi.

```
@Composable
fun FlowDemoScreen() {
    val scope = rememberCoroutineScope()
    val logs = remember { mutableStateListOf<String>() }
    var latestValue by remember { mutableStateOf( value = 0) }
    val listState = rememberLazyListState()

    LaunchedEffect( key1 = logs.size) {
        if (logs.isNotEmpty()) {
            listState.animateScrollToItem( index = logs.size - 1)
        }
    }

    Column(...) {
        Text(...)
        Spacer(Modifier.height( height = 20.dp))

        Card(modifier = Modifier.padding( all = 16.dp)) {...}

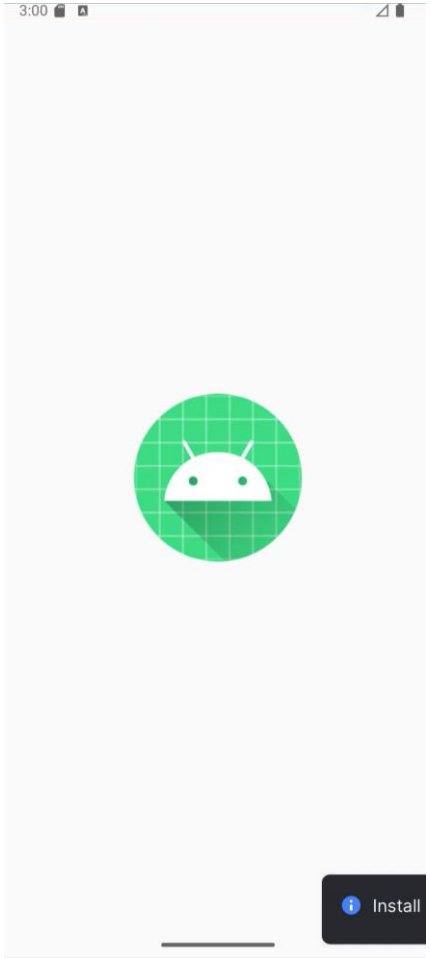
        Button(
            onClick = {
                scope.launch {
                    logs.add("--- NOWA SUBSKRYPCJA ---")
                    numberStream(logs).collect { number ->
                        latestValue = number
                    }
                }
            },
            modifier = Modifier.fillMaxWidth()
        ) {...}
```

Uniwersytet Wrocławski

StateFlow

Flow jest świetny do **jednorazowych operacji**, które mają zwrócić wiele wyników (np. pobieranie pliku i emitowanie postępu w procentach). Ale **nie nadaje się** do przechowywania **stanu UI**, ponieważ każdy nowy obserwator (np. po obrocie ekranu) **uruchamiałby go od nowa**.

StateFlow to specjalny, ***"gorący"*** strumień, zaprojektowany do przechowywania i udostępniania stanu.



StateFlow

W przeciwieństwie do "*zimnego*" Flow, który jest tylko przepisem, "*gorący*" strumień musi być **aktywnie zarządzany**.



Licznik widzów na pulpicie streamera. **Zawsze ma jakąś wartość** (zaczyna od 0)

```
class TwitchStreamer {
    private val _viewerCount = MutableStateFlow(0)
    val viewerCount: StateFlow<Int> = _viewerCount.asStateFlow()

    init {
        GlobalScope.launch {
            while (true) {
                _viewerCount.value = Random.nextInt(100, 1000)
                delay(2000)
            }
        }
    }
}
```

StateFlow

W przeciwieństwie do "*zimnego*" Flow, który jest tylko przepisem, "*gorący*" strumień musi być **aktywnie zarządzany**.



konwertuje modyfikowalny MutableStateFlow na jego publiczną, niemodyfikowalną (tylko do odczytu) wersję StateFlow.

Jest to mechanizm ochronny, który uniemożliwia modyfikację stanu z zewnątrz klasy, która jest jego właścicielem (zazwyczaj z ViewModelu).

Licznik widzów na pulpicie streamera. **Zawsze ma jakąś wartość** (zaczyna od 0)

```
class TwitchStreamer {
    private val _viewerCount = MutableStateFlow(0)
    val viewerCount: StateFlow<Int> = _viewerCount.asStateFlow()

    init {
        GlobalScope.launch {
            while (true) {
                _viewerCount.value = Random.nextInt(100, 1000)
                delay(2000)
            }
        }
}
```

wzorzec z _viewerCount i viewerCount zapewnia **niemodyfikowalność** (*read-only*) stanu dla warstwy UI, co jest niemożliwe do osiągnięcia przy użyciu private set na MutableStateFlow.

private set chroni tylko przed przypisaniem nowego obiektu MutableStateFlow do zmiennej viewerCount z zewnątrz. Jednak UI (np. Composable) wciąż otrzymuje obiekt typu MutableStateFlow. Oznacza to, że wciąż może nadpisać wartość.

StateFlow

W przeciwieństwie do "*zimnego*" Flow, który jest tylko przepisem, "*gorący*" strumień musi być **aktywnie zarządzany**.



konwertuje modyfikowalny MutableStateFlow na jego publiczną, niemodyfikowalną (tylko do odczytu) wersję StateFlow.

Jest to mechanizm ochronny, który uniemożliwia modyfikację stanu z zewnątrz klasy, która jest jego właścicielem (zazwyczaj z ViewModelu).

Licznik widzów na pulpicie streamera. **Zawsze ma jakąś wartość** (zaczyna od 0)

```
class TwitchStreamer {
    private val _viewerCount = MutableStateFlow(0)
    val viewerCount: StateFlow<Int> = _viewerCount.asStateFlow()

    init {
        GlobalScope.launch {
            while (true) {
                _viewerCount.value = Random.nextInt(100, 1000)
                delay(2000)
            }
        }
    }
```

wzorzec z _viewerCount i viewerCount zapewnia **niemodyfikowalność** (*read-only*) stanu dla warstwy UI, co jest niemożliwe do osiągnięcia przy użyciu private set na MutableStateFlow.

private set chroni tylko przed przypisaniem nowego obiektu MutableStateFlow do zmiennej viewerCount z zewnątrz. Jednak UI (np. Composable) wciąż otrzymuje obiekt typu MutableStateFlow. Oznacza to, że wciąż może nadpisać wartość.

Gdy tylko tworzony jest obiekt TwitchStreamer, natychmiast uruchamia w tle korutynę, która w pętli symuluje zmiany liczby widzów. Transmisja ruszyła i jest "*na żywo*".

GlobalScope jest używany tylko dla uproszczenia tego przykładu.

StateFlow

Przekazany zakres określa czas życia symulacji.

```
class TwitchStreamer(scope: CoroutineScope) {
    private val _viewerCount = MutableStateFlow(0)
    val viewerCount = _viewerCount.asStateFlow()

    init {
        scope.launch {
            while (isActive) {
                delay(2_000)
                _viewerCount.value =
                    Random.nextInt(100, 1_000)
            }
        }
    }
}
```

asStateFlow() udostępnia odczyt. Zmiany wykonuje właściciel _viewerCount.

StateFlow

Każdy widz zaczyna od aktualnego licznika.




```
fun main() = runBlocking {
    val streamer = TwitchStreamer(this)
    delay(5_000) // Producent działa bez widzów.

    launch {
        streamer.viewerCount.collect {
            println("Widz 1: $it")
        }
    }
    delay(4_000)
    launch {
        streamer.viewerCount.collect {
            println("Widz 2: $it")
        }
    }
    delay(4_000)
    coroutineContext.cancelChildren()
}
```

StateFlow

Każdy widz zaczyna od aktualnego licznika.



Streamer rozpoczyna transmisję. Jego licznik widzów już zaczyna się zmieniać

```
fun main() = runBlocking {
    val streamer = TwitchStreamer(this)
    delay(5_000) // Producent działa bez widzów.

    launch {
        streamer.viewerCount.collect {
            println("Widz 1: $it")
        }
    }
    delay(4_000)
    launch {
        streamer.viewerCount.collect {
            println("Widz 2: $it")
        }
    }
    delay(4_000)
    coroutineContext.cancelChildren()
}
```

StateFlow

Każdy widz zaczyna od aktualnego licznika.



Streamer rozpoczyna transmisję. Jego licznik widzów już zaczyna się zmieniać

Transmisja trwa, mimo że nikt jej nie subskrybuje.

```
fun main() = runBlocking {
    val streamer = TwitchStreamer(this)
    delay(5_000) // Producent działa bez widzów.

    launch {
        streamer.viewerCount.collect {
            println("Widz 1: $it")
        }
    }
    delay(4_000)
    launch {
        streamer.viewerCount.collect {
            println("Widz 2: $it")
        }
    }
    delay(4_000)
    coroutineContext.cancelChildren()
}
```

StateFlow

Każdy widz zaczyna od aktualnego licznika.




```
fun main() = runBlocking {
    val streamer = TwitchStreamer(this)
    delay(5_000) // Producent działa bez widzów.

    launch {
        streamer.viewerCount.collect {
            println("Widz 1: $it")
        }
    }
    delay(4_000)
    launch {
        streamer.viewerCount.collect {
            println("Widz 2: $it")
        }
    }
    delay(4_000)
    coroutineContext.cancelChildren()
```

Streamer rozpoczyna transmisję. Jego licznik widzów już zaczyna się zmieniać

Transmisja trwa, mimo że nikt jej nie subskrybuje.

Pierwszy widz wchodzi na kanał. Nie widzi licznika od 0. Natychmiast dostaje **aktualną wartość licznika**, np. 458.

StateFlow

Każdy widz zaczyna od aktualnego licznika.




```
fun main() = runBlocking {
    val streamer = TwitchStreamer(this)
    delay(5_000) // Producent działa bez widzów.

    launch {
        streamer.viewerCount.collect {
            println("Widz 1: $it")
        }
    }
    delay(4_000)
    launch {
        streamer.viewerCount
            println("Widz 2:
        }
    }
    delay(4_000)
    coroutineContext.cancelChildren()
```

Streamer rozpoczyna transmisję. Jego licznik widzów już zaczyna się zmieniać

Transmisja trwa, mimo że nikt jej nie subskrybuje.

Pierwszy widz wchodzi na kanał. Nie widzi licznika od 0. Natychmiast dostaje **aktualną wartość licznika**, np. 458.

Drugi widz wchodzi na kanał. On również natychmiast dostaje aktualną wartość, np. 721, i od tego momentu **obaj widzowie widzą te same, synchroniczne aktualizacje**.

StateFlow

StateFlow: jest "***gorący***", **zawsze** ma **aktualny stan**, który natychmiast **udostępnia nowym subskrybentom**. **Nie jest historią wszystkich zmian.**




```
fun main() = runBlocking {
    val streamer = TwitchStreamer(this)
    delay(5_000) // Producent działa bez widzów.

    launch {
        streamer.viewerCount.collect {
            println("Widz 1: $it")
        }
    }
    delay(4_000)
    launch {
        streamer.viewerCount
            println("Widz 2:
        }
    }
    delay(4_000)
    coroutineContext.cancelChildren()
```

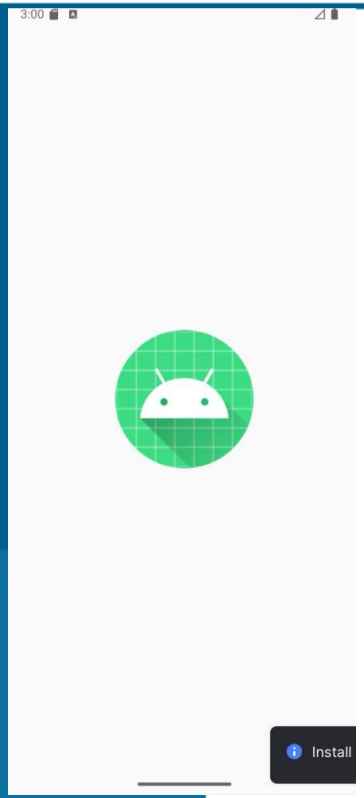
Streamer rozpoczyna transmisję. Jego licznik widzów już zaczyna się zmieniać

Transmisja trwa, mimo że nikt jej nie subskrybuje.

Pierwszy widz wchodzi na kanał. Nie widzi licznika od 0. Natychmiast dostaje **aktualną wartość licznika**, np. 458.

Drugi widz wchodzi na kanał. On również natychmiast dostaje aktualną wartość, np. 721, i od tego momentu **obaj widzowie widzą te same, synchroniczne aktualizacje**.

StateFlow




```
class HotStreamViewModel : ViewModel() {
    // Prywatny, modyfikowalny StateFlow - licznik, który widzi tylko streamer.
    2 Usages
    private val _numberStream = MutableStateFlow( value = 0)
    // Publiczny, niemodyfikowalny StateFlow - to widzi publiczność.
    3 Usages
    val numberStream: StateFlow<Int> = _numberStream.asStateFlow()

    init {
        // Streamer rozpoczyna "transmisję na żywo" w momencie utworzenia ViewModelu.
        // Strumień jest "gorący" - działa niezależnie od liczby widzów (subskrybentów).
        viewModelScope.launch {
            var count = 0
            while (true) {
                delay( timeMillis = 1000)
                count++
                _numberStream.value = count
            }
        }
    }
}
1 Usage
@Composable
fun StateFlowDemoScreen(viewModel: HotStreamViewModel = viewModel()) {
    val liveValue by viewModel.numberStream.collectAsStateWithLifecycle()
```

viewModelScope to gotowy **CoroutineScope**, który jest **wbudowany** w klasę ViewModel z biblioteki AndroidX Lifecycle. Jego głównym zadaniem jest uruchamianie korutyn, których **cykl życia** jest **automatycznie powiązany** z **cyklem życia ViewModelu**.

collectAsStateWithLifecycle to bezpieczniejsza i zoptymalizowana wersja collectAsState, która **wstrzymuje zbieranie danych ze strumienia**, gdy aplikacja **działa w tle** (jest w stanie STOPPED) i wznawia je, gdy wraca na pierwszy plan.

Uniwersytet Wrocławski

Kolektor i Producent

Zbieranie stanu UI oraz produkowanie danych mają różne czasy życia.

collectAsStateWithLifecycle()

Domyślnie zbiera przy co najmniej STARTED. Poniżej tego progu kończy zbieranie.

Pętla w viewModelScope

Może działać dalej, gdy UI nie zbiera danych. Kończy się przy usunięciu ViewModelu.

Powrót na ekran

UI ponownie subskrybuje StateFlow i otrzymuje bieżący stan.

SharedFLow

Wiemy już, że StateFlow jest jak główny ekran transmisji na Twitchu – **zawsze** pokazuje **aktualny stan** transmisji, liczbę widzów itp. Nowy widz, który dołącza, od razu widzi ten stan.

Co się dzieje w sytuacji, gdy streamer dostanie donację? Na ekranie pojawia się animacja i dźwięk, a na czacie wyskakuje wiadomość. To jest **ZDARZENIE**, a **nie STAN**. Nie chcemy, żeby nowy widz, który dołączy do transmisji 5 minut później, nagle zobaczył animację tej starej donacji. To **zdarzenie jest ulotne** i przeznaczone tylko dla tych, którzy oglądali w danym momencie.

Donacja jest zdarzeniem: uruchamia animację lub dźwięk. W tej demonstracji alert otrzymują tylko obecni odbiorcy. Nowy widz nie odtwarza wcześniejszych alertów.



Uniwersytet Wrocławski

SharedFLow

Wiemy już, że StateFlow jest jak główny ekran transmisji na Twitchu – **zawsze** pokazuje **aktualny stan** transmisji, liczbę widzów itp. Nowy widz, który dołącza, od razu widzi ten stan.

Co się dzieje w sytuacji, gdy streamer dostanie donację? Na ekranie pojawia się animacja i dźwięk, a na czacie wyskakuje wiadomość. To jest **ZDARZENIE**, a **nie STAN**. Nie chcemy, żeby nowy widz, który dołączy do transmisji 5 minut później, nagle zobaczył animację tej starej donacji. To **zdarzenie jest ulotne** i przeznaczone tylko dla tych, którzy oglądali w danym momencie.

Do obsługi takich jednorazowych zdarzeń służy **SharedFlow** (replay = 0). To również jest ***'gorący'*** strumień, ale działający na innych zasadach niż StateFlow



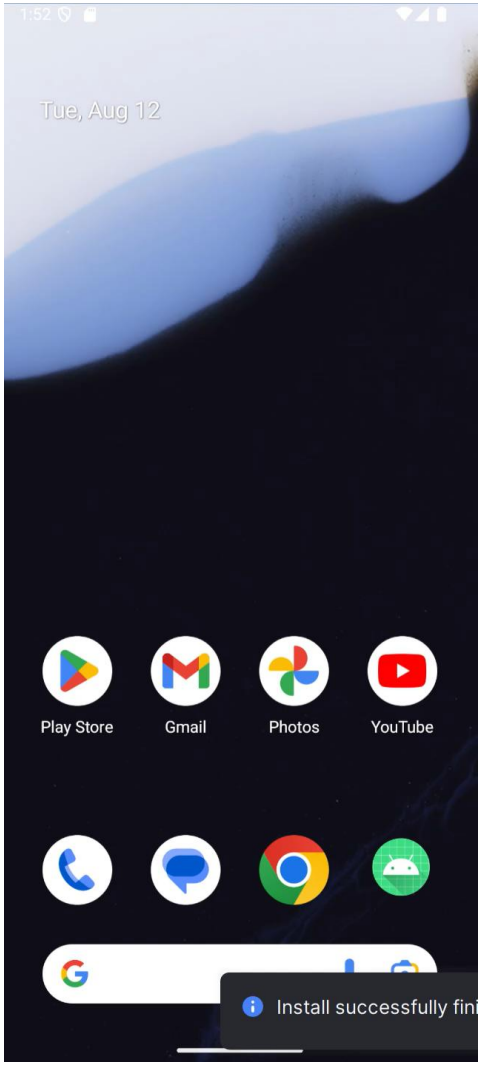
SharedFLow

SharedFlow współdzieli wartości między odbiorcami. W tej demonstracji używamy replay = 0.

Główny ekran transmisji. **Zawsze** jest widoczny, **zawsze** ma jakąś wartość (**stan**).

Powiadomienia o subskrypcji, donacji lub wiadomości na czacie. Są to **jednorazowe** "*strzały*" informacji. Pojawiają się, widzą je **wszyscy**, **którzy akurat patrzą**, i **znikają**. Nie są częścią trwałego stanu.





Uniwersytet Wrocławski

SharedFLow

Stan transmisji i zdarzenia mają osobne strumienie.

```
data class Donation(
    val id: Int,
    val donor: String,
    val amount: Int
)

class StreamerViewModel(replay: Int = 0) : ViewModel() {
    private val _viewerCount = MutableStateFlow(0)
    val viewerCount = _viewerCount.asStateFlow()
    private val _donations =
        MutableSharedFlow<Donation>(replay = replay)
    val donations = _donations.asSharedFlow()
    val subscriberCount = _donations.subscriptionCount
```



SharedFLow

Dalsza część klasy StreamerViewModel.

```
private var nextId = 0

    init {
        viewModelScope.launch {
            while (isActive) {
                delay(2_000)
                _viewerCount.value =
                    Random.nextInt(100, 1_000)
            }
        }
    }

    fun sendDonation(donor: String, amount: Int) {
        val donation = Donation(++nextId, donor, amount)
        viewModelScope.launch {
            _donations.emit(donation)
        }
    }
}
```



Uniwersytet Wrocławski

SharedFlow: kto otrzyma donację?



Ten sam scenariusz wykonujemy dla replay = 0 i replay = 1.

Moment	replay = 0	replay = 1
Emisja przed dołączeniem	Brak odbiorców	Zapis w replay cache
Dołącza widz 1	Czeka na nowe zdarzenie	Dostaje poprzednie
Emisja „Anna”	Odbiera widz 1	Odbiera widz 1
Dołącza widz 2	Nie dostaje „Anny”	Dostaje „Annę”
Emisja „Piotr”	Odbierają obaj	Odbierają obaj

SharedFLow



Czekamy na subskrypcję zamiast zgadywać ją przez delay().

```
fun main() = runBlocking {
    val events = MutableSharedFlow<String>(replay = 0)
    events.emit("Przed dołączeniem")
    val first = launch {
        events.collect { println("Widz 1: $it") }
    }
    events.subscriptionCount.first { it == 1 }
    events.emit("Anna")

    val second = launch {
        events.collect { println("Widz 2: $it") }
    }
    events.subscriptionCount.first { it == 2 }
    events.emit("Piotr")
    first.cancelAndJoin()
    second.cancelAndJoin()
}
```

SharedFlow: replay i bufor



Parametry odpowiadają na różne pytania.

Parametr	Znaczenie
replay	Ile wcześniejszych wartości otrzyma nowy subskrybent?
extraBufferCapacity	Ile dodatkowych wartości może czekać na wolnych, obecnych odbiorców?
onBufferOverflow	Co zrobić, gdy bufor się zapełni: zawiesić emitera czy odrzucić wartość?

Bez subskrybentów pamiętamy tylko replay. Dodatkowy bufor nie tworzy historii.

Uniwersytet Wrocławski

SharedFlow: emit i tryEmit



Powodzenie emisji nie oznacza potwierdzenia obsługi przez UI.

Operacja	Zachowanie
emit(value)	Funkcja suspend. Może czekać na miejsce lub odbiór przez subskrybentów.
tryEmit(value)	Wraca od razu. false oznacza, że emisja wymagałaby zawieszenia.
Brak odbiorców	Przy replay = 0 tryEmit może zwrócić true, mimo że wartość przepada.

Nie zastępujemy emit przez tryEmit bez określenia zasad utraty zdarzeń.

Uniwersytet Wrocławski

SharedFlow: emit i tryEmit



Powodzenie emisji nie oznacza potwierdzenia obsługi przez UI.

Operacja	Zachowanie
emit(value)	Funkcja suspend. Może czekać na miejsce lub odbiór przez subskrybentów.
tryEmit(value)	Wraca od razu. false oznacza, że emisja wymagałaby zawieszenia.
Brak odbiorców	Przy replay = 0 tryEmit może zwrócić true, mimo że wartość przepada.

Nie zastępujemy emit przez tryEmit bez określenia zasad utraty zdarzeń.

Podsumowanie

Dobór strumienia wynika z oczekiwanego zachowania.

Narzędzie	Co reprezentuje?	Nowy odbiorca
flow { ... }	Zimny strumień. Oddzielne wykonanie dla każdego collect.	Uruchamia własne wykonanie źródła.
StateFlow	Bieżący stan z wartością początkową. Równe wartości nie emitują ponownie.	Otrzymuje aktualny stan.
SharedFlow	Wspólny gorący strumień. Replay i bufor są konfigurowalne.	Otrzymuje replay, a potem nowe wartości.

Uniwersytet Wrocławski

Przykład zastosowania

